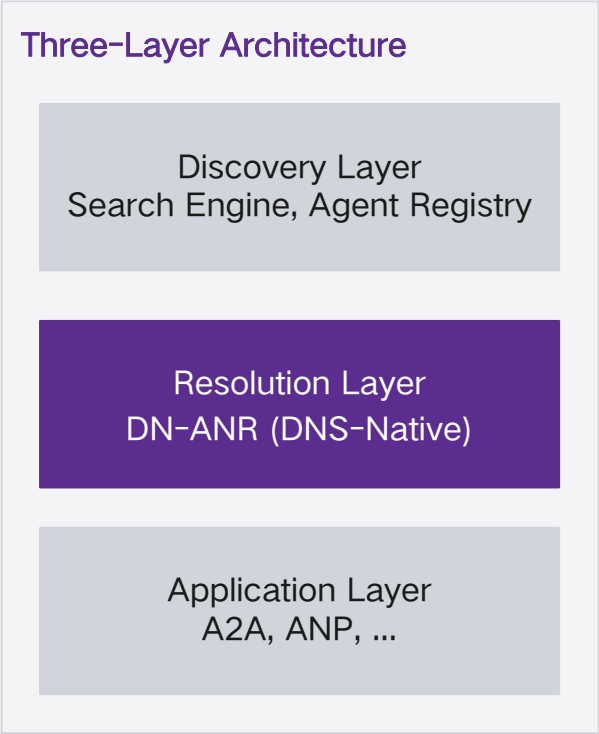
AI Agent Naming, Resolution, and Discovery

A Joint IETF Draft Presentation

Draft 1: draft-cui-dns-native-agent-naming-resolution
Draft 2: draft-cui-ai-agent-discovery-invocation



Yong Cui (Tsinghua University) · **Yihan Chao** · Chenguang Du (Zhongguancun Laboratory)

Motivation: The AI Agent Interoperability Problem

Why do we need standardized agent naming and discovery?

No Stable Identity

Each platform assigns agents local identifiers. There is no globally stable, administratively managed name that uniquely identifies an agent across organizations, platforms, or time.

Fragmented Discovery

Agents are registered in isolated silos. Clients wishing to use agents from multiple vendors must adapt to disparate, incompatible APIs and metadata schemas — significantly increasing integration cost.

No Trusted Resolution

Even when an agent ID is known, there is no secure, deterministic way to resolve it to a verifiable endpoint with cryptographic integrity guarantees. Discovery, resolution, and invocation are mixed together.

Our approach: Separate concerns into distinct layers. Let DNS handle naming and resolution; let dedicated protocols handle discovery and application invocation.

A Three-Layer Architecture for AI Agent Communication

Clear separation of concerns: Discovery → Resolution → Application

To address the interoperability problem, we propose a layered architecture where each layer has a clearly defined scope and interface. The two IETF drafts presented here cover different layers of this architecture, and are designed to work together seamlessly.

Discovery Layer

Search engines, agent registries, semantic routers, DNS-SD/mDNS
OUT OF SCOPE for DN-ANR — handled by AIDIP (Draft 2)

Candidate Agent Identifiers (FQDNs)

Resolution Layer — DN-ANR (Draft 1)

Agent FQDN → endpoint, protocol/version, cryptographic integrity material
Built on DNS SVCB/HTTPS RRs (RFC 9460) + DNSSEC

Verifiable endpoint, protocol, version

Application Layer

A2A, ANP, MCP, HTTPS, gRPC — protocol-specific session logic
OUT OF SCOPE for DN-ANR

Key Design Principle

DN-ANR acts as the resolution layer — the glue between discovery and application. It takes a selected Agent Identifier and securely resolves it to connection material.

The Discovery Layer (Draft 2 — AIDIP) finds candidate agents via registries and semantic search. It operates above DN-ANR.

The Application Layer executes the actual protocol (A2A, ANP) after receiving the verified endpoint from DN-ANR.

This strict layering avoids coupling, improves security, and enables each layer to evolve independently.

Draft 1: DNS-Native Agent Naming and Resolution (DN-ANR)

draft-cui-dns-native-agent-naming-resolution

DN-ANR defines how to use the Domain Name System as the authoritative source for AI agent identity and endpoint resolution. It specifies three concrete goals, each carefully scoped to avoid overloading DNS with heavy semantic metadata.

Goal 1: Identity Naming

Use domain names (FQDNs) as stable Agent Identifiers. Domain ownership provides administrative authority — no central registration is required. Agent version changes do not create new identities. Example: translator.agents.example.com

Goal 2: Trusted Resolution & Connection Guidance

Given an Agent Identifier, securely resolve it to an endpoint, protocol declarations, and version information. Cryptographic integrity is provided via DNSSEC (recommended) or TXT-based signatures (optional). Clients get verifiable connection material, not just an address.

Goal 3: Minimal Support for Discovery Systems

Expose only stable, minimal pointers and indexes (e.g., descriptor URIs via TXT records) that upper-layer discovery systems may reference. DN-ANR deliberately keeps heavy capability metadata out of DNS to preserve DNS efficiency and stability.

Non-Goals: DN-ANR does NOT perform semantic matching, capability ranking, cross-domain discovery, or task routing. It only resolves a selected identifier.

DNS Record Design: Three Record Types

Keep DNS simple and lightweight — minimal, stable payloads

Three DNS Records

SVCB — Main information

- TargetName, Port — Agent endpoint
- key65480 — Agent version: v1, v3.2
- key65481 — Agent protocol: a2a, anp
- ipv4hint, ipv6hint — Pre–fetched address

A/AAAA — Basic Connectivity
for clients that don't support SVCB

TXT — External metadata
description

Keep DNS simple and lightweight

SVCB

```
_agent.translator.example.com. IN SVCB 1 (
 agent-v3.example.com.
 alpn=h2 port=443
 ipv4hint=203.0.113.50 ; hints
 ipv6hint=2001:db8::50
 key65480="v3" ; version
 key65481="a2a,anp" ; protocol )
```

A/AAAA

- Default agent endpoint
- (Optional) .well-known JSON mirror of DNS records

TXT

```
_agent.translator.example.com. IN TXT (
 alg=Ed25519; pk=...; sig=...;  ; Optional, record integrity verification
 svcb-digest=...;               ; Optional, record integrity verification
 agent-desc=...; agent-desc-sha256=...; ; Optional, pointing to metadata
 " )
```

Security Model and HTTPS Fallback

Two complementary mechanisms for integrity and authenticity

DN-ANR provides two complementary security mechanisms. They can be used independently or together for defense in depth, depending on deployment context.

DNSSEC — Recommended for Internet-facing Agents

DNSSEC provides protocol-level cryptographic authentication for all DNS record sets — including SVCB, TXT, and A/AAAA. Trust is anchored to the DNS root zone. When a SVCB RRSet validates as bogus, clients MUST treat resolution as failure (fail-closed). This protects against DNS cache poisoning and MITM attacks on the resolution path.

Signature-based Security — Optional but Recommended

Agents publish a public key (pk) and signature (sig) in their TXT record. Two key options: (1) use the domain's TLS certificate keys — enabling verification via Web PKI trust chain; or (2) use separate signing keys. A svcb-digest field optionally cross-checks SVCB record integrity. This is especially useful for enterprise or private networks where DNSSEC is not deployed.

HTTPS Fallback: agent-dns.json

For clients that do not support SVCB queries (e.g., browsers, debugging tools), agents publish a JSON mirror of their DNS records at https://{agent-id}/.well-known/agent-dns.json. This file mirrors SVCB and TXT content and MUST be signed using the same key material. DNS remains the authoritative source — JSON is a readable fallback only, not a new control plane. Clients MUST verify the JSON signature before consuming the endpoint data. DNS-first principle: if SVCB resolves successfully, HTTPS fallback should not be used.

Draft 2: AI Agent Discovery and Invocation Protocol (AIDIP)

draft-cui-ai-agent-discovery-invocation-01

Agent Metadata Specification

- Structured JSON Schema for agent identity, capabilities, input/output schemas, and authentication requirements.
- Core fields: id, name, description, version, publisher, capabilities[], tags[], endpoint, operations[].
- Each operation defines its name, description, and JSON Schema for inputs/outputs with optional examples.
- Supports both machine-readable discovery and human understanding of what an agent does.

Capability-Based Discovery

- Agent Registry aggregates metadata; supports attribute-based filtering by capabilities, tags, languages.
- POST /agents/search with JSON filters; returns ranked candidate agents with essential summary fields.
- Optional semantic search: client submits a natural language query; registry uses AI-based embedding matching.
- Agent Semantic Resolution (ASR) extension: resolve task intent to candidate agents before invocation.

Unified RESTful Invocation

- POST to agent endpoint URL with JSON body conforming to the agent's published input schema.
- Standard HTTP status codes: 200 OK for success, 4xx for client errors, 5xx for agent/server errors.
- Standardized error object format: {"error": {"code": "...", "message": "..."}}.
- Supports streaming responses, batch requests, and idempotency considerations for robust operation.

Interaction flow: Search registry → Select candidate → GET full metadata via DN-ANR → POST invocation with auth → Handle response.

Agent Metadata Format and Invocation Interface

Concrete schema example: Chinese-English Translation Agent

```
{
  "id": "agent-12345",
  "name": "Chinese-English Translator",
  "version": "1.2.0",
  "capabilities": ["translation"],
  "tags": ["nlp","chinese","english"],
  "endpoint": "https://api.example.com/translate",
  "supported_languages": ["en","zh"],
  "authentication": {
    "type": "api_key"
  },
  "operations": [{
    "name": "translateText",
    "inputs": {
      "text": {"type":"string"},
      "source_language": ...,
      "target_language": ...
  }}]}
```

Invocation Flow

1. **Discovery**
   Client queries registry: POST /agents/search with {"capabilities":["translation"], "supported_languages":["zh","en"]}. Registry returns candidate list.

2. **Resolution via DN-ANR**
   Client resolves the agent FQDN (e.g., translator.agents.example.com) via DNS SVCB to get the verified endpoint, protocol, and version.

3. **Invocation**
   Client POSTs to endpoint with JSON input body and X-API-Key header. Response: {"translated_text": "Hello World"}.

Agent Semantic Resolution (ASR) and Protocol Comparison

Optional AIDIP extension — intent-based agent selection

AIDIP introduces Agent Semantic Resolution (ASR) as an optional extension that enables intent-based agent selection before standard discovery and invocation. This is particularly useful for Host Agents or orchestrators that describe tasks in natural language.

ASR Conceptual Model

A Semantic Routing Platform (SRP) implements ASR. It answers: "Which agent(s) should handle this task now?" — not "Where is agent X located?" (that is DNS's job). ASR is fully optional; existing clients continue to use standard discovery and invocation. ASR does not replace DNS-based resolution.

ASR Non-Goals

- Does NOT provide name-to-address resolution
- Does NOT maintain global persistent agent identifiers
- Does NOT replace DNS, ANS, or URI-based registries
- Does NOT participate in task execution
- Is fully backward compatible with base AIDIP

How DN-ANR and AIDIP Complement Each Other

Aspect	DN-ANR (Draft 1)	AIDIP (Draft 2)
Layer	Resolution	Discovery + Application
Input	One selected Agent FQDN	Task intent / capability query
Output	Verified endpoint + protocol + version	Candidate agent list + invocation result
DNS usage	Core (SVCB, TXT, A/AAAA)	References DN-ANR for endpoint resolution
Security	DNSSEC + TXT signatures	TLS + OAuth 2.0 / API keys

Summary

Toward a standardized AI agent communication infrastructure

DN-ANR — What We Proposed

Use domain names as stable, administratively managed Agent Identifiers. Use DNS SVCB records to resolve an identifier to endpoint, protocol, and version. Use DNSSEC or TXT signatures for cryptographic integrity. Provide an HTTPS fallback mirror for broad client compatibility. Keep DNS simple: no heavy metadata, no semantic matching.

AIDIP — What We Proposed

Define a standard JSON metadata schema for agent description (capabilities, I/O, authentication). Define a RESTful Agent Registry for attribute-based and semantic discovery. Define a uniform invocation interface (POST + JSON) with standardized error responses. Add optional Agent Semantic Resolution (ASR) for intent-based selection without breaking existing clients.